

SISTEMA INTEGRADOR DE PARQUEADERO

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Construya una aplicación completa utilizando el siguiente menú:

PARQUEADERO UNIVERSITARIO

1. Registrar vehículo
2. Mostrar vehículos registrados
3. Mostrar estadísticas
4. Mostrar recaudación
5. Salir

Por cada vehículo registrar:

- Tipo de vehículo.
- Rol: estudiante, docente o visitante.
- Número de horas.
- Día de la semana.
- Boleto perdido: sí/no.

El programa deberá aplicar diferentes tarifas definidas por el equipo. El reporte final deberá mostrar:

- Vehículos registrados.
- Cantidad por tipo.
- Cantidad por rol.
- Total de horas.
- Promedio de permanencia.
- Total recaudado.
- Mayor valor pagado.
- Menor valor pagado.

Debe integrar obligatoriamente: for + while + do-while + switch + ciclos anidados + contadores + acumuladores + validaciones + casos límite.

Tarifas y criterios definidos para esta solución

El enunciado permite al equipo definir las tarifas. Para este ejercicio se propone la siguiente tabla; son valores académicos, no tarifas oficiales de una universidad.

Concepto	Valor definido
Motocicleta / automóvil / camioneta	\$0,50 / \$1,00 / \$1,50 por hora
Estudiante / docente / visitante	Descuento del 20 % / 10 % / 0 %
Lunes a viernes / sábado / domingo	Factor 1,00 / 1,20 / 1,50
Boleto perdido	Recargo fijo de \$5,00, después del descuento
Permanencia permitida	Horas enteras de 1 a 24
Almacenamiento de la sesión	Máximo 50 registros; no se eliminan registros

El límite de 50 registros es una decisión técnica para trabajar con arreglos estáticos de primer semestre. No representa el número de espacios físicos libres. Los datos se conservan durante la ejecución; al cerrar el programa no se guardan en un archivo.

Orden del cobro

1. Base = tarifa por hora $\times$ horas.
2. Importe por día = base $\times$ factor del día.
3. Importe con descuento = importe por día $\times$ (1 - descuento del rol).
4. Redondear ese importe al centavo más cercano.
5. Si se perdió el boleto, agregar \$5,00.

En los códigos se trabaja con centavos enteros para sumar exactamente los cobros. Se usa factor = 100, 120 o 150 y descuento = 20, 10 o 0. La operación es:

base = tarifaCentavos * horas;

pago = Trunc((base * factor * (100 - descuento) + 5000) / 10000);

Si boleto = 1: pago = pago + 500;

La suma de 5000 permite redondear al centavo antes de truncar. Un empate de medio centavo se redondea hacia arriba. El recargo fijo no recibe descuento. Por ejemplo, $\$0,50 \times 1,50 \times 0,90 = \$0,675$ se cobra como \$0,68.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- opcion: entero de 1 a 5 según el menú.
- tipo: entero; 1 = motocicleta, 2 = automóvil, 3 = camioneta.
- rol: entero; 1 = estudiante, 2 = docente, 3 = visitante.
- horas: entero de 1 a 24, incluidos los límites.
- dia: entero de 1 a 7; lunes = 1, martes = 2, miércoles = 3, jueves = 4, viernes = 5, sábado = 6 y domingo = 7.
- boleto: entero; 0 = no perdido, 1 = perdido.
- Todos los campos se escriben con dígitos, sin espacios, signos ni decimales. Se rechazan letras, entradas vacías y números fuera del rango.

1.2 Proceso

- Inicializar n, totalHoras, total, mayor y menor en cero. Reservar la matriz datos[50][5] y el arreglo pagos[50].
- Mostrar el menú con do-while; validar la opción con while y seleccionar la acción con switch.
- Para registrar, comprobar que $n < 50$. Si se alcanzó la capacidad, informar y regresar al menú sin leer ni guardar otro vehículo.
- Leer y validar los cinco datos del vehículo. Obtener tarifa, factor y descuento de las tablas; calcular el cobro en centavos y aplicar el recargo cuando corresponda.
- Guardar datos y pago, acumular horas y recaudación; actualizar mayor y menor. En el primer registro, ambos toman el pago recibido. Aumentar n en uno y confirmar el registro.
- En el listado, recorrer solo los n registros con for y mostrar sus datos y pagos. Si $n = 0$, informar que no existen vehículos.
- En estadísticas, usar un for por categoría y un for interno por registro. Contar coincidencias de tipo y de rol; mostrar totalHoras y calcular $\text{totalHoras} / n$ solo si $n > 0$.
- En recaudación, mostrar total, mayor y menor convertidos de centavos a dólares. Sin registros, el total es cero y los extremos no están disponibles.
- Al elegir 5, presentar listado, estadísticas y recaudación como reporte final; después terminar el ciclo y cerrar el programa.

1.3 Salida de datos

- Menú, solicitudes, mensajes de dato inválido y de capacidad alcanzada.
- Confirmación con número de registro y valor a pagar.
- Listado con tipo, rol, horas, día, boleto perdido y pago.
- Cantidad total de registros, cantidades por tipo y por rol, horas totales y promedio de permanencia.
- Total recaudado, pago mayor y pago menor; mensajes cuando no hay datos.
- Reporte final al salir y mensaje de finalización.

2. ALGORITMO

1. Inicio.

2. Declarar las variables, la matriz de 50 filas y 5 columnas y el arreglo de 50 pagos. Inicializar los contadores y acumuladores en cero.

3. Mostrar las cinco opciones y leer una opción entera válida. Repetir la lectura mientras no esté entre 1 y 5.

4. Seleccionar la acción de acuerdo con la opción.

5. Si la opción es 1 y existen 50 registros, mostrar el mensaje de capacidad alcanzada y continuar en el paso 16.

6. Si la opción es 1 y hay espacio, leer tipo, rol, horas, día y boleto; repetir cada lectura hasta que sea válida.

7. Seleccionar la tarifa por tipo, el factor por día y el descuento por rol según la tabla definida.

8. Calcular $\text{base} = \text{tarifa} \times \text{horas}$ y $\text{pago} = \text{Trunc}((\text{base} \times \text{factor} \times (100 - \text{descuento}) + 5000) / 10000)$. Si el boleto está perdido, sumar 500 centavos.

9. Guardar los cinco datos y el pago en la posición n. Sumar las horas a totalHoras y el pago a total.

10. Si $n = 0$ o $\text{pago} > \text{mayor}$, actualizar mayor. Si $n = 0$ o $\text{pago} < \text{menor}$, actualizar menor. Aumentar n en uno y mostrar la confirmación. Continuar en el paso 16.

11. Si la opción es 2, mostrar las leyendas y recorrer los registros del 0 al $n - 1$ para mostrar sus datos y pagos; si $n = 0$, mostrar que no hay registros. Continuar en el paso 16.

12. Si la opción es 3, mostrar n. Para cada categoría de 1 a 3, poner porTipo y porRol en cero y recorrer todos los registros: aumentar los contadores cuando corresponda. Mostrar ambos conteos por categoría.

13. En la opción 3, mostrar totalHoras y, si $n > 0$, el promedio $\text{totalHoras} / n$; de lo contrario, informar que no está disponible. Continuar en el paso 16.

14. Si la opción es 4, mostrar $\text{total} / 100$; si hay registros, mostrar $\text{mayor} / 100$ y $\text{menor} / 100$; si no, informar que no están disponibles. Continuar en el paso 16.

15. Si la opción es 5, ejecutar las consultas de los pasos 11 a 14 para mostrar el reporte final completo y el mensaje de finalización.

16. Si la opción es diferente de 5, regresar al paso 3; de lo contrario, cerrar el programa.

17. Fin.

El promedio y los valores monetarios se presentan con dos decimales. Los contadores se reinician al consultar estadísticas para evitar que una segunda consulta duplique los resultados.

3. PSEUDOCÓDIGO

Compatible con un perfil de PSeInt que habilite subprocessos, funciones de cadenas e índices en base 0. Copiar todos los subprocessos y el algoritmo principal en un mismo archivo. La matriz utiliza filas 0 a 49 y columnas 0 a 4. PSeInt puede omitir ceros decimales finales al mostrar importes; los valores calculados son los mismos.

```
SubProceso valor <- LeerEntero(mensaje, minimo, maximo)
  Definir mensaje, texto, caracter Como Cadena;
  Definir minimo, maximo, valor, k Como Entero;
  Definir valido, digito Como Logico;
  valido <- Falso;
  Mientras No valido Hacer
    Escribir Sin Saltar mensaje;
    Leer texto;
    valor <- 0;
    k <- 0;
    valido <- Longitud(texto) > 0;
    Mientras k < Longitud(texto) Y valido Hacer
      caracter <- Subcadena(texto, k, k);
      digito <- caracter = "0" O caracter = "1";
      digito <- digito O caracter = "2" O caracter = "3";
      digito <- digito O caracter = "4" O caracter = "5";
      digito <- digito O caracter = "6" O caracter = "7";
      digito <- digito O caracter = "8" O caracter = "9";
      Si digito Entonces
        valor <- valor * 10 + ConvertirANumero(caracter);
        Si valor > maximo Entonces
          valido <- Falso;
        FinSi
      SiNo
        valido <- Falso;
      FinSi
      k <- k + 1;
    FinMientras
    valido <- valido Y valor >= minimo Y valor <= maximo;
    Si No valido Entonces
      Escribir "Dato invalido. Intente nuevamente.";
    FinSi
  FinMientras
FinSubProceso
```

```
SubProceso MostrarVehiculos(datos Por Referencia, pagos Por Referencia, n)
  Definir i, n Como Entero;
  Escribir "VEHICULOS REGISTRADOS";
  Escribir "Tipos: 1 Moto, 2 Auto, 3 Camioneta.";
  Escribir "Roles: 1 Estudiante, 2 Docente, 3 Visitante.";
  Escribir "Dias: 1 Lunes a 7 Domingo. Boleto: 0 No, 1 Si.";
  Si n = 0 Entonces
    Escribir "No hay vehiculos registrados.";
  FinSi
```

Pseudocódigo — continuación

```
Para i <- 0 Hasta n - 1 Hacer
    Escribir "Registro ", i + 1, " | Tipo ", datos[i,0];
    Escribir "Rol ", datos[i,1], " | Horas ", datos[i,2];
    Escribir "Dia ", datos[i,3], " | Boleto perdido ", datos[i,4];
    Escribir "Pago: $", pagos[i] / 100;
FinPara
FinSubProceso

SubProceso MostrarEstadisticas(datos Por Referencia, n, totalHoras)
    Definir n, totalHoras, categoria, i, porTipo, porRol Como Entero;
    Escribir "ESTADISTICAS";
    Escribir "Vehiculos registrados: ", n;
    Escribir "Tipos: 1 Moto, 2 Auto, 3 Camioneta.";
    Escribir "Roles: 1 Estudiante, 2 Docente, 3 Visitante.";
    Para categoria <- 1 Hasta 3 Hacer
        porTipo <- 0;
        porRol <- 0;
        Para i <- 0 Hasta n - 1 Hacer
            Si datos[i,0] = categoria Entonces
                porTipo <- porTipo + 1;
            FinSi
            Si datos[i,1] = categoria Entonces
                porRol <- porRol + 1;
            FinSi
        FinPara
        Escribir "Tipo ", categoria, ": ", porTipo;
        Escribir "Rol ", categoria, ": ", porRol;
    FinPara
    Escribir "Total de horas: ", totalHoras;
    Si n > 0 Entonces
        Escribir "Promedio: ", Redon(totalHoras / n * 100) / 100;
    SiNo
        Escribir "Promedio de permanencia: no disponible.";
    FinSi
FinSubProceso

SubProceso MostrarRecaudacion(n, total, mayor, menor)
    Definir n, total, mayor, menor Como Entero;
    Escribir "RECAUDACION";
    Escribir "Total recaudado: $", total / 100;
    Si n > 0 Entonces
        Escribir "Mayor pago: $", mayor / 100;
        Escribir "Menor pago: $", menor / 100;
    SiNo
        Escribir "Mayor y menor pago: no disponibles.";
    FinSi
FinSubProceso

Algoritmo ParqueaderoIntegrador
    Definir datos, pagos Como Entero;
    Dimension datos[50,5], pagos[50];
    Definir n, totalHoras, total, mayor, menor Como Entero;
```

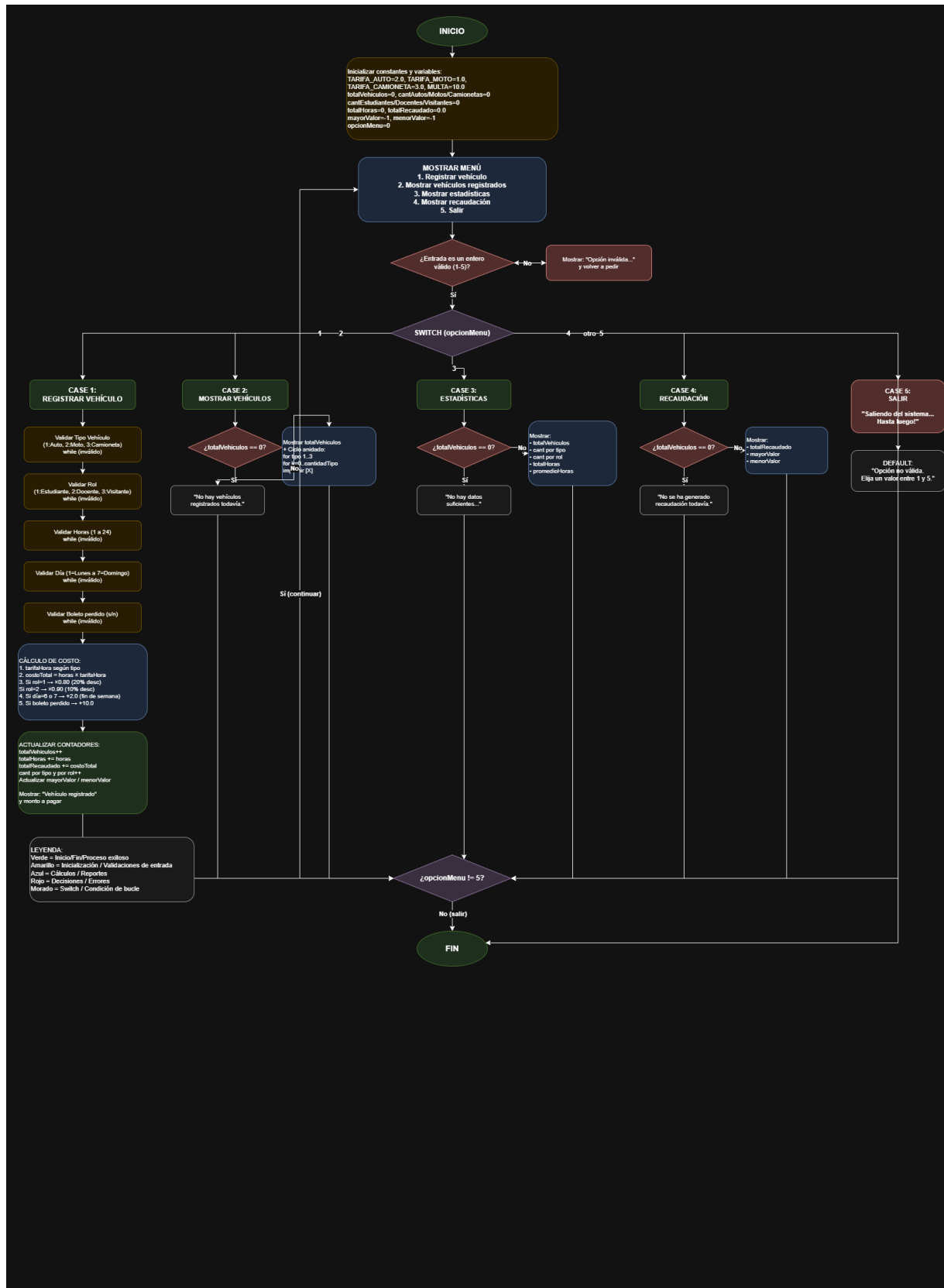
Pseudocódigo — continuación

```
Definir opcion, tipo, rol, horas, dia, boleto Como Entero;
Definir tarifa, factor, descuento, base, pago Como Entero;
n <- 0;
totalHoras <- 0;
total <- 0;
mayor <- 0;
menor <- 0;
Repetir
    Escribir "PARQUEADERO UNIVERSITARIO";
    Escribir "1. Registrar vehiculo";
    Escribir "2. Mostrar vehiculos registrados";
    Escribir "3. Mostrar estadisticas";
    Escribir "4. Mostrar recaudacion";
    Escribir "5. Salir";
    opcion <- LeerEntero("Opcion: ", 1, 5);
    Segun opcion Hacer
        1:
            Si n = 50 Entonces
                Escribir "Capacidad de 50 registros alcanzada.";
            SiNo
                tipo <- LeerEntero("Tipo: 1 Moto, 2 Auto, 3 Camioneta: ", 1, 3);
                rol <- LeerEntero("Rol: 1 Estudiante, 2 Docente, 3 Visitante: ", 1, 3);
                horas <- LeerEntero("Horas (1 a 24): ", 1, 24);
                dia <- LeerEntero("Dia (1 Lunes a 7 Domingo): ", 1, 7);
                boleto <- LeerEntero("Boleto perdido (0 No, 1 Si): ", 0, 1);
                Segun tipo Hacer
                    1: tarifa <- 50;
                    2: tarifa <- 100;
                    3: tarifa <- 150;
                FinSegun
                factor <- 100;
                Si dia = 6 Entonces
                    factor <- 120;
                FinSi
                Si dia = 7 Entonces
                    factor <- 150;
                FinSi
                descuento <- 0;
                Si rol = 1 Entonces
                    descuento <- 20;
                FinSi
                Si rol = 2 Entonces
                    descuento <- 10;
                FinSi
                base <- tarifa * horas;
                pago <- Trunc((base * factor * (100 - descuento) + 5000) / 10000);
                Si boleto = 1 Entonces
                    pago <- pago + 500;
                FinSi
                datos[n,0] <- tipo;
                datos[n,1] <- rol;
                datos[n,2] <- horas;
```

Pseudocódigo — continuación

```
    datos[n,3] <- dia;
    datos[n,4] <- boleto;
    pagos[n] <- pago;
    totalHoras <- totalHoras + horas;
    total <- total + pago;
    Si n = 0 O pago > mayor Entonces
        mayor <- pago;
    FinSi
    Si n = 0 O pago < menor Entonces
        menor <- pago;
    FinSi
    n <- n + 1;
    Escribir "Registro ", n, " guardado. Pago: $", pago / 100;
FinSi
2:
    MostrarVehiculos(datos, pagos, n);
3:
    MostrarEstadisticas(datos, n, totalHoras);
4:
    MostrarRecaudacion(n, total, mayor, menor);
5:
    Escribir "REPORTE FINAL";
    MostrarVehiculos(datos, pagos, n);
    MostrarEstadisticas(datos, n, totalHoras);
    MostrarRecaudacion(n, total, mayor, menor);
    Escribir "Fin del programa.";
FinSegun
Hasta Que opcion = 5
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1. Sistema vacío y salida inmediata

Se ejecutan las opciones 2, 3, 4 y 5 antes de registrar vehículos. Se verifica que no haya división para cero ni mínimos o máximos inventados.

Variable	Valor
Opciones de entrada	2 → 3 → 4 → 5
n	0
totalHoras / total	0 horas / 0 centavos
porTipo (1, 2, 3)	0, 0, 0
porRol (1, 2, 3)	0, 0, 0
mayor / menor internos	0 / 0; no representan pagos reales
Promedio de permanencia	No disponible
Total recaudado	\$0,00
Mayor y menor pago mostrados	No disponibles
Resultado final	Reporte vacío y finalización normal

Operaciones paso a paso

1. $n = 0$; no se ejecuta el recorrido de registros.
2. Para cada categoría, $\text{porTipo} = 0$ y $\text{porRol} = 0$; el ciclo interno no realiza iteraciones.
3. $\text{totalHoras} = 0$ y $\text{total} = 0$; no se agregan datos.
4. $n > 0$ es falso: no se calcula $0 / 0$.
5. Total en dólares = $0 / 100 = \$0,00$. No existe un valor mayor ni menor pagado.
6. Opción 5 presenta el reporte y termina el do-while.

Caso 2. Tres vehículos con tarifas y roles diferentes

Se registran tres vehículos y después se consultan las opciones 2, 3, 4 y 5. Este caso comprueba descuentos, días laborales, sábado, domingo, boleto perdido y acumulación.

Variable	Valor
V1: tipo, rol, horas, día, boleto	2; 1; 3; 1; 0 (auto, estudiante, lunes)
V2: tipo, rol, horas, día, boleto	3; 2; 2; 6; 1 (camioneta, docente, sábado)
V3: tipo, rol, horas, día, boleto	1; 3; 1; 7; 0 (moto, visitante, domingo)
Pagos finales	V1: \$2,40; V2: \$8,24; V3: \$0,75
n / totalHoras	3 / 6
total / mayor / menor	1139 / 824 / 75 centavos
Cantidad por tipo (1, 2, 3)	1, 1, 1
Cantidad por rol (1, 2, 3)	1, 1, 1
Promedio / recaudación	2,00 horas / \$11,39

Registro	Pago (\$)	Horas acum.	Total (\$)	Mayor / menor (\$)
V1	2,40	3	2,40	2,40 / 2,40
V2	8,24	5	10,64	8,24 / 2,40
V3	0,75	6	11,39	8,24 / 0,75

Operaciones paso a paso

V1: base = $\$1,00 \times 3 = \$3,00$. Día: $\$3,00 \times 1,00 = \$3,00$. Descuento: $\$3,00 \times 0,20 = \$0,60$. Pago: $\$3,00 - \$0,60 = \$2,40$. Sin recargo. En centavos: $\text{Trunc}((300 \times 100 \times 80 + 5000) / 10000) = \text{Trunc}(240,5) = 240$.

V2: base = $\$1,50 \times 2 = \$3,00$. Sábado: $\$3,00 \times 1,20 = \$3,60$. Descuento: $\$3,60 \times 0,10 = \$0,36$. Importe: $\$3,60 - \$0,36 = \$3,24$. Boleto perdido: $\$3,24 + \$5,00 = \$8,24$. En centavos: $\text{Trunc}((300 \times 120 \times 90 + 5000) / 10000) + 500 = 324 + 500 = 824$.

V3: base = $\$0,50 \times 1 = \$0,50$. Domingo: $\$0,50 \times 1,50 = \$0,75$. Descuento y recargo: $\$0,00$. En centavos: $\text{Trunc}((50 \times 150 \times 100 + 5000) / 10000) = 75$.

Total de horas = $3 + 2 + 1 = 6$. Promedio = $6 / 3 = 2$ horas. Recaudación = $240 + 824 + 75 = 1139$ centavos = $\$11,39$. Mayor = $\$8,24$; menor = $\$0,75$. Las cantidades por tipo y por rol suman 3 cada una.

Caso 3. Datos inválidos y límite de 24 horas

Se ingresan datos incorrectos antes de los válidos. El registro final corresponde a una motocicleta de visitante durante 24 horas en domingo, con boleto perdido.

Variable	Valor
opcion	9 inválido → 1 válido; luego 5
tipo	4 inválido → 1
rol	0 inválido → 3
horas	0 y 25 inválidos → 24
dia	8 inválido → 7
boleto	2 inválido → 1
Errores de entrada	7 rechazos; no producen registros
tarifa / factor / descuento	50 centavos / 150 % / 0 %
base / pago	1200 / 2300 centavos
n / totalHoras	1 / 24
Cantidad por tipo (1, 2, 3)	1, 0, 0
Cantidad por rol (1, 2, 3)	0, 0, 1
Promedio	24,00 horas
Total / mayor / menor	\$23,00 / \$23,00 / \$23,00

Operaciones paso a paso

1. Los siete intentos inválidos solo generan mensajes y nuevas lecturas; n permanece en 0 hasta completar el registro.
2. Base = $\$0,50 \times 24 = \$12,00$.
3. Domingo = $\$12,00 \times 1,50 = \$18,00$.
4. Visitante: descuento = 0 %; importe = \$18,00.
5. En centavos: $\text{Trunc}((1200 \times 150 \times 100 + 5000) / 10000) = 1800$.
6. Boleto perdido: $1800 + 500 = 2300$ centavos = \$23,00.
7. Primer registro: mayor = menor = 2300; n = 0 + 1 = 1.
8. totalHoras = 0 + 24 = 24; total = 0 + 2300 = 2300.
9. Promedio = $24 / 1 = 24$ horas. El reporte final muestra un solo vehículo.

Resumen de los casos

Caso	Registros	Horas / prom.	Recaudación	Mayor / menor
1. Vacío	0	0 / no disponible	\$0,00	No disponibles
2. Variados	3	6 / 2,00	\$11,39	\$8,24 / \$0,75
3. Validaciones	1	24 / 24,00	\$23,00	\$23,00 / \$23,00

6. CÓDIGO

JAVA

```
import java.util.Scanner;
import java.util.Locale;

public class ParqueroIntegrador {
    // Repetimos la lectura mientras el entero no sea valido.
    static int leerEntero(Scanner sc, String mensaje, int min, int max) {
        String texto;
        double valor;
        boolean valido = false;
        while (!valido) {
            System.out.print(mensaje);
            texto = sc.nextLine();
            if (texto.matches("[0-9]+")) {
                valor = Double.parseDouble(texto);
                if (valor >= min && valor <= max) {
                    return (int) valor;
                }
            }
            System.out.println("Dato invalido. Intente nuevamente.");
        }
        return 0;
    }

    static void mostrarVehiculos(int[][] datos, int[] pagos, int n) {
        int i;
        System.out.println("VEHICULOS REGISTRADOS");
        System.out.println("Tipos: 1 Moto, 2 Auto, 3 Camioneta.");
        System.out.println("Roles: 1 Estudiante, 2 Docente, 3 Visitante.");
        System.out.println("Dias: 1 Lunes a 7 Domingo. Boleto: 0 No, 1 Si.");
        if (n == 0) {
            System.out.println("No hay vehiculos registrados.");
        }
        for (i = 0; i < n; i++) {
            System.out.printf(Locale.US,
                "Registro %d | Tipo %d | Rol %d | Horas %d%n",
                i + 1, datos[i][0], datos[i][1], datos[i][2]);
            System.out.printf(Locale.US,
                "Dia %d | Boleto perdido %d | Pago: $%.2f%n",
                datos[i][3], datos[i][4], pagos[i] / 100.0);
        }
    }

    static void mostrarEstadisticas(int[][] datos, int n, int totalHoras) {
```

```

int categoria, i, porTipo, porRol;
System.out.println("ESTADISTICAS");
System.out.println("Vehiculos registrados: " + n);
System.out.println("Tipos: 1 Moto, 2 Auto, 3 Camioneta.");
System.out.println("Roles: 1 Estudiante, 2 Docente, 3 Visitante.");
// Ciclos anidados: revisamos los registros de cada categoria.
for (categoria = 1; categoria <= 3; categoria++) {
    porTipo = 0;
    porRol = 0;
    for (i = 0; i < n; i++) {
        if (datos[i][0] == categoria) { porTipo++; }
        if (datos[i][1] == categoria) { porRol++; }
    }
    System.out.println("Tipo " + categoria + ": " + porTipo);
    System.out.println("Rol " + categoria + ": " + porRol);
}
System.out.println("Total de horas: " + totalHoras);
if (n > 0) {
    System.out.printf(Locale.US,
        "Promedio de permanencia: %.2f horas%n", totalHoras / (double) n);
} else {
    System.out.println("Promedio de permanencia: no disponible.");
}
}

static void mostrarRecaudacion(int n, int total, int mayor, int menor) {
    System.out.println("RECAUDACION");
    System.out.printf(Locale.US, "Total recaudado: $%.2f%n", total / 100.0);
    if (n > 0) {
        System.out.printf(Locale.US, "Mayor pago: $%.2f%n", mayor / 100.0);
        System.out.printf(Locale.US, "Menor pago: $%.2f%n", menor / 100.0);
    } else {
        System.out.println("Mayor y menor pago: no disponibles.");
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    final int CAPACIDAD = 50;
    int[][] datos = new int[CAPACIDAD][5];
    int[] pagos = new int[CAPACIDAD];
    int n = 0, totalHoras = 0, total = 0, mayor = 0, menor = 0;
    int opcion, tipo, rol, horas, dia, boleto;
    int tarifa, factor, descuento, base, pago;

    do {
        System.out.println("PARQUEADERO UNIVERSITARIO");
        System.out.println("1. Registrar vehiculo");
        System.out.println("2. Mostrar vehiculos registrados");
        System.out.println("3. Mostrar estadisticas");
        System.out.println("4. Mostrar recaudacion");
        System.out.println("5. Salir");
        opcion = leerEntero(sc, "Opcion: ", 1, 5);
        switch (opcion) {

```

```

case 1:
    if (n == CAPACIDAD) {
        System.out.println("Capacidad de 50 registros alcanzada.");
        break;
    }
    tipo = leerEntero(sc, "Tipo (1 Moto, 2 Auto, 3 Camioneta): ", 1, 3);
    rol = leerEntero(sc, "Rol (1 Estudiante, 2 Docente, 3 Visitante): ", 1, 3);
    horas = leerEntero(sc, "Horas (1 a 24): ", 1, 24);
    dia = leerEntero(sc, "Dia (1 Lunes a 7 Domingo): ", 1, 7);
    boleto = leerEntero(sc, "Boleto perdido (0 No, 1 Si): ", 0, 1);
    tarifa = tipo == 1 ? 50 : (tipo == 2 ? 100 : 150);
    factor = dia <= 5 ? 100 : (dia == 6 ? 120 : 150);
    descuento = rol == 1 ? 20 : (rol == 2 ? 10 : 0);
    base = tarifa * horas;
    // Trabajamos en centavos y redondeamos al centavo mas cercano.
    pago = (base * factor * (100 - descuento) + 5000) / 10000;
    if (boleto == 1) { pago += 500; }
    datos[n][0] = tipo;
    datos[n][1] = rol;
    datos[n][2] = horas;
    datos[n][3] = dia;
    datos[n][4] = boleto;
    pagos[n] = pago;
    totalHoras += horas;
    total += pago;
    if (n == 0 || pago > mayor) { mayor = pago; }
    if (n == 0 || pago < menor) { menor = pago; }
    n++;
    System.out.printf(Locale.US,
        "Registro %d guardado. Pago: $%.2f%n", n, pago / 100.0);
    break;
case 2:
    mostrarVehiculos(datos, pagos, n);
    break;
case 3:
    mostrarEstadisticas(datos, n, totalHoras);
    break;
case 4:
    mostrarRecaudacion(n, total, mayor, menor);
    break;
case 5:
    System.out.println("REPORTE FINAL");
    mostrarVehiculos(datos, pagos, n);
    mostrarEstadisticas(datos, n, totalHoras);
    mostrarRecaudacion(n, total, mayor, menor);
    System.out.println("Fin del programa.");
    break;
    }
} while (opcion != 5);
sc.close();
}
}

```

C++

```
#include <iostream>
#include <iomanip>
#include <string>
#include <sstream>
#include <regex>
using namespace std;

// Repetimos la lectura mientras el entero no sea valido.
int leerEntero(string mensaje, int min, int max) {
    string texto;
    double valor;
    bool valido = false;
    while (!valido) {
        cout << mensaje;
        getline(cin, texto);
        if (regex_match(texto, regex("[0-9]+"))) {
            istringstream lectura(texto);
            if (lectura >> valor) {
                if (valor >= min && valor <= max) {
                    return int(valor);
                }
            }
        }
        cout << "Dato invalido. Intente nuevamente." << endl;
    }
    return 0;
}

void mostrarVehiculos(int datos[][5], int pagos[], int n) {
    int i;
    cout << "VEHICULOS REGISTRADOS" << endl;
    cout << "Tipos: 1 Moto, 2 Auto, 3 Camioneta." << endl;
    cout << "Roles: 1 Estudiante, 2 Docente, 3 Visitante." << endl;
    cout << "Dias: 1 Lunes a 7 Domingo. Boleto: 0 No, 1 Si." << endl;
    if (n == 0) { cout << "No hay vehiculos registrados." << endl; }
    for (i = 0; i < n; i++) {
        cout << "Registro " << i + 1 << " | Tipo " << datos[i][0]
            << " | Rol " << datos[i][1] << " | Horas " << datos[i][2] << endl;
        cout << "Dia " << datos[i][3] << " | Boleto perdido " << datos[i][4]
            << " | Pago: $" << pagos[i] / 100.0 << endl;
    }
}

void mostrarEstadisticas(int datos[][5], int n, int totalHoras) {
    int categoria, i, porTipo, porRol;
    cout << "ESTADISTICAS" << endl;
    cout << "Vehiculos registrados: " << n << endl;
```

```

cout << "Tipos: 1 Moto, 2 Auto, 3 Camioneta." << endl;
cout << "Roles: 1 Estudiante, 2 Docente, 3 Visitante." << endl;
// Ciclos anidados: revisamos los registros de cada categoria.
for (categoria = 1; categoria <= 3; categoria++) {
    porTipo = 0;
    porRol = 0;
    for (i = 0; i < n; i++) {
        if (datos[i][0] == categoria) { porTipo++; }
        if (datos[i][1] == categoria) { porRol++; }
    }
    cout << "Tipo " << categoria << ": " << porTipo << endl;
    cout << "Rol " << categoria << ": " << porRol << endl;
}
cout << "Total de horas: " << totalHoras << endl;
if (n > 0) {
    cout << "Promedio de permanencia: " << totalHoras / double(n)
        << " horas" << endl;
} else {
    cout << "Promedio de permanencia: no disponible." << endl;
}
}

void mostrarRecaudacion(int n, int total, int mayor, int menor) {
    cout << "RECAUDACION" << endl;
    cout << "Total recaudado: $" << total / 100.0 << endl;
    if (n > 0) {
        cout << "Mayor pago: $" << mayor / 100.0 << endl;
        cout << "Menor pago: $" << menor / 100.0 << endl;
    } else {
        cout << "Mayor y menor pago: no disponibles." << endl;
    }
}

int main() {
    const int CAPACIDAD = 50;
    int datos[CAPACIDAD][5], pagos[CAPACIDAD];
    int n = 0, totalHoras = 0, total = 0, mayor = 0, menor = 0;
    int opcion, tipo, rol, horas, dia, boleto;
    int tarifa, factor, descuento, base, pago;
    cout << fixed << setprecision(2);

    do {
        cout << "PARQUEADERO UNIVERSITARIO" << endl;
        cout << "1. Registrar vehiculo" << endl;
        cout << "2. Mostrar vehiculos registrados" << endl;
        cout << "3. Mostrar estadisticas" << endl;
        cout << "4. Mostrar recaudacion" << endl;
        cout << "5. Salir" << endl;
        opcion = leerEntero("Opcion: ", 1, 5);
        switch (opcion) {
            case 1:
                if (n == CAPACIDAD) {
                    cout << "Capacidad de 50 registros alcanzada." << endl;
                    break;
                }

```

```

    }
    tipo = leerEntero("Tipo (1 Moto, 2 Auto, 3 Camioneta): ", 1, 3);
    rol = leerEntero("Rol (1 Estudiante, 2 Docente, 3 Visitante): ", 1, 3);
    horas = leerEntero("Horas (1 a 24): ", 1, 24);
    dia = leerEntero("Dia (1 Lunes a 7 Domingo): ", 1, 7);
    boleto = leerEntero("Boleto perdido (0 No, 1 Si): ", 0, 1);
    tarifa = tipo == 1 ? 50 : (tipo == 2 ? 100 : 150);
    factor = dia <= 5 ? 100 : (dia == 6 ? 120 : 150);
    descuento = rol == 1 ? 20 : (rol == 2 ? 10 : 0);
    base = tarifa * horas;
    // Trabajamos en centavos y redondeamos una sola vez.
    pago = (base * factor * (100 - descuento) + 5000) / 10000;
    if (boleto == 1) { pago += 500; }
    datos[n][0] = tipo;
    datos[n][1] = rol;
    datos[n][2] = horas;
    datos[n][3] = dia;
    datos[n][4] = boleto;
    pagos[n] = pago;
    totalHoras += horas;
    total += pago;
    if (n == 0 || pago > mayor) { mayor = pago; }
    if (n == 0 || pago < menor) { menor = pago; }
    n++;
    cout << "Registro " << n << " guardado. Pago: $"
         << pago / 100.0 << endl;
    break;
case 2:
    mostrarVehiculos(datos, pagos, n);
    break;
case 3:
    mostrarEstadisticas(datos, n, totalHoras);
    break;
case 4:
    mostrarRecaudacion(n, total, mayor, menor);
    break;
case 5:
    cout << "REPORTE FINAL" << endl;
    mostrarVehiculos(datos, pagos, n);
    mostrarEstadisticas(datos, n, totalHoras);
    mostrarRecaudacion(n, total, mayor, menor);
    cout << "Fin del programa." << endl;
    break;
}
} while (opcion != 5);
return 0;
}

```